

せいきひょうげん 正規表現

プログラミングを勉強していると必ず出会う、魔法の言葉のような記号の羅列。それが「正規表現」です。英語で "Regular Expression" と言います。

JavaScriptでは、**RegExp**（レグエクスプ）オブジェクトを使います。

RegExpは「魔法の虫めがね」です。たくさんの文字の中から、自分の探したい「パターン」だけをピカッと光らせて見つけることができます。

I. 「正規表現」ってなに？

普通の検索（`indexOf` など）は、「特定の言葉」を探します。

RegExpは、「言葉のパターン（ルール）」を探します。

- 普通の検索：「『田中』さんを探して」
- **RegExp**：「『数字が3つ、ハイフン、数字が4つ』の形（電話番号）を探して」

このように、あやふやなルールで検索できるのがすごいところです。

II. RegExpオブジェクトの作り方

2つの書き方がありますが、よく使われるのは `/ /`（スラッシュ）で囲む方法です。

```
// 1. スラッシュで囲む（リテラル） ★おすすめ
const pattern1 = /abc/;

// 2. new を使う
const pattern2 = new RegExp("abc");
```

ただし、このドキュメントでは、一般的なオブジェクト作成の書き方である、`new` 演算子を使います。

III. よく使う「魔法の記号」（1）

正規表現（RegExp）のメタ文字は、文字の「種類」や「数」を指定する特別なルールです。

「トランプのジョーカー」や「算数のルール」に例えて説明すると、パズルのように楽しく理解できます。

よく使われる7つの記号(メタ文字)を、やさしく解説します！

1. . (ドット)

→ 「なんでもいい一文字」

- イメージ：トランプの「ジョーカー」や、クイズの「穴埋（あなう）め」です。
- ルール：数字でも、漢字でも、アルファベットでも、**「とにかく1文字」**あればOK。

例： /あ.い/

- 「あい」 ... OK
- 「ああい」 ... OK
- 「あ1い」 ... OK
- 「あい」 ... NG（文字が足りない）

2. * (アスタリスク)

→ 「0回以上のくり返し」

- イメージ：おかわり自由。「なくてもいいし、無限（むげん）にあってもいい」。
- ルール：直前（ちよくぜん）の文字が「0個、1個、2個.....」と、いくつ続いていてもOK。

例： /あ*い/

- 「い」 ... OK（「あ」が0個でもいい）
- 「あい」 ... OK
- 「あああい」 ... OK

3. + (プラス)

→ 「1回以上のくり返し」

- イメージ：最低（さいてい）でも1つは必要。「1個以上なら、いくつでもいい」。
- ルール：直前の文字が「1個、2個、3個.....」と続いていればOK。

例： /あ+い/

- 「い」 ... **NG**（「あ」が最低1つは必要！）
- 「あい」 ... OK
- 「あああい」 ... OK

4. ? (クエスション)

→ 「0回か 1回だけ」

- イメージ：「あってもなくてもいい（最大1つまで）」。

- **ルール**：直前の文字が「ない」か「1個だけある」状態。

例：/あ?い/

- 「い」 ... OK
- 「あい」 ... OK
- 「ああい」 ... **NG** (2個以上はダメ)

5. ^ (キャレット)

→ 「行がここから始まる」

- **イメージ**：マラソンの「スタートライン」です。
- **ルール**：その文字で文章が始まっていることをチェックします。

例：/^おはよう/

- 「おはようございます」 ... OK
- 「みんな、おはよう」 ... **NG** (「みんな」から始まっているから)

6. \$ (ドル)

→ 「行の最後がこれでおわる」

- **イメージ**：マラソンの「ゴールテープ」です。
- **ルール**：その文字で文章が終わっていることをチェックします。

例：/です\$/

- 「田中です」 ... OK
- 「田中ですよ」 ... **NG** (「よ」で終わっているから)

7. | (パイプ・縦線)

→ 「または (OR)」

- **イメージ**：レストランのメニュー。「A または B どちらか選んで」というルール。
- **ルール**：左側か右側、どちらかの文字があればOK。

例：/りんご|みかん/

- 「りんご」 ... OK
- 「みかん」 ... OK
- 「バナナ」 ... NG

きごう 記号	い み 意味	おほ かた 覚え方のヒント
.	なんでも1文字	何が入るかワクワクする「穴」
*	0個以上	おかわり自由（なくてもいい）
+	1個以上	必ず1つは持ってきて！
?	0か1個	オプション（あってもなくてもいい）
^	始まり	文の最初にあるスタートライン
\$	終わり	文の最後にあるゴールテープ
	または	AかB、どちらか好きな方！

いかがでしょうか？記号の「パワー」を知ると、短いコードで複雑なチェックができるようになります。

4. ? (クエスチョン)

→ 「0回か 1回だけ」

- イメージ：「あってもなくてもいい（最大1つまで）」。
- ルール：直前の文字が「ない」か「1個だけある」状態。

例：/あ?い/

- 「い」 ... OK
- 「あい」 ... OK
- 「ああい」 ... NG (2個以上はダメ)

5. ^ (キャレット)

→ 「行がここから始まる」

- イメージ：マラソンの「スタートライン」です。
- ルール：その文字で文章が始まっていることをチェックします。

例：/^おはよう/

- 「おはようございます」 ... OK
- 「みんな、おはよう」 ... NG (「みんな」から始まっているから)

6. \$ (ドル)

→ 「行の最後がこれでおわる」

- イメージ：マラソンの「ゴールテープ」です。
- ルール：その文字で文章が終わっていることをチェックします。

れい
例： /です\$/

- 「田中^{たなか}です」 ... OK
- 「田中^{たなか}ですよ」 ... **NG**（「よ^お」で終わっているから）

7. |（パイプ・縦線）

→「または（OR）」

- **イメージ**：レストランのメニュー。「A **または** B どちらか^{えら}選んで」というルール。
- **ルール**：左側^{ひだりがわ}か右側^{みぎがわ}、どちらかの文字^{もじ}があればOK。

れい
例： /りんご|みかん/

- 「りんご」 ... OK
- 「みかん」 ... OK
- 「バナナ」 ... NG

覚え方のまとめテーブル

きごう 記号	いみ 意味	おぼ かた 覚え方のヒント
.	なんでも1文字 ^{もじ}	何 ^{なに} が入 ^{はい} るかワクワクする「穴」 ^{あな}
*	0個以上 ^{こいじょう}	おかわり自由 ^{じゆう} （なくてもいい）
+	1個以上 ^{こいじょう}	必ず 1つは持 ^も ってきて！
?	0か1個 ^こ	オプション（あってもなくてもいい）
^	始まり ^{はじ}	文 ^{ぶん} の最初 ^{さいしょ} にあるスタートライン
\$	終わり ^お	文 ^{ぶん} の最後 ^{さいご} にあるゴールテープ
	または	AかB、どちらか好 ^す きな方 ^{ほう} ！

いかがでしょうか？ 記号^{きごう}の「パワー」を知^しると、短^{みじか}いコードで複雑^{ふくざつ}なチェックができるようになります。

IV. よく使う「魔法の記号」（2）

せいぎひょうげん
正規表現（RegExp）のメタ文字^{もじ}、さらに便利^{べんり}なものを紹^{しょうかい}介^{かい}します！ 今^{こんかい}回は、文字^{もじ}を「グルー^かープ化」したり、
かいすう せいかく してい
「回数^{かいすう}を正確^{せいかく}に指^{してい}定^{すこ}」したりする、少^{すこ}し高^{こう}度^どなルールです。

「チー^{つく}ム作^くり」や「回^{かいすう}数の^{よやく}予^た約^{せつめい}」に例^{たと}えて説^{せつめい}明^{めい}します。

1. ()（かっこ）

→「チー^{つく}ム（グルー^かープ）を作^{つく}る」

- **イメージ**：複数の文字をひとつにまとめて「1つのチーム」として扱います。
- **ルール**：チームに対して、さっき勉強した * や + を使うことができます。

例：/(abc)+/

- 「abc」 ... OK
- 「abcbcabcb」 ... OK（「abc」というチームが繰り返されている）
- 「abcc」 ... NG（チーム全員で繰り返さないとダメ）

2. []（角かっこ）

→ 「この中のどれか1文字」

- **イメージ**：「この中（リスト）にある文字なら、誰でもいいよ」というメンバー名簿です。
- **ルール**：カッコの中に入っている文字のどれか1つに合えばOK。

例：/[ABC]<ん/

- 「A<ん」 ... OK
- 「B<ん」 ... OK
- 「C<ん」 ... OK
- 「D<ん」 ... NG（名簿に名前がない）

このように[]で囲まれて『この中のどれか1つに合えばOK』という文字のグループを**文字クラス**といいます。

文字クラスにはいくつかのバリエーションがあります。

- **文字クラスの先頭に^をつけると、文字クラスの否定を表します。**

例：/[^ABC]<ん/

- 「A<ん」 ... NG
- 「B<ん」 ... NG
- 「C<ん」 ... NG
- 「D<ん」 ... OK（名簿に名前がないものがOK）

- **数字やアルファベットが連続しているとき、最初の文字と最後の文字を-でつなげて、短く書くことができます。**

例：/[A-C]<ん/

- 「A<ん」 ... OK
- 「B<ん」 ... OK
- 「C<ん」 ... OK
- 「D<ん」 ... NG（AからCのあいだにない文字）

3. {n}{n,}{n, m} (波かっこ)

→ 「回数^{かいすう}を数字^{すうじ}で指定^{してい}する」

これらは + や * よりも、もっと細^{こま}かく「何回^{なんかい}か」を予約^{よやく}する記号^{きごう}です。

{n} : ぴったり n回^{かい}

- 例^{れい} : /\d{3}/ (数字^{すうじ}がぴったり3つ)
- 「123」はOK、「12」はNG。

{n,} : n回^{かい}以上 (いくらでも)

- 例^{れい} : /\d{3,}/ (数字^{すうじ}が3つ以上^{いじょう})
- 「123」も「12345」もOK。

{n, m} : n回^{かい}から m回^{かい}のあいだ

- 例^{れい} : /\d{3,5}/ (数字^{すうじ}が3つ、4つ、または5つ)
- 「1234」はOK、「123456」はNG。

4. \ (バックスラッシュ / 円^{えん}マーク)

→ 「魔法^{まほう}を消^けして、普通^{ふつう}の文字^{もじ}に戻^{もど}す」

- イメージ : 「エスケープ (逃^にがす)」といいます。特別^{とくべつ}な力^{ちから}を持^もっているメタ文字^{もじ}から、魔法^{まほう}を取りあ^とげます。
- ルール : メタ文字^{もじ}の前^{まえ}に書^かくと、ただの「記号^{きごう}」として探^{さが}せるようになります。

例^{れい} : 普通^{ふつう}のドット (.) を探^{さが}したいとき

- . だけだと「なんでも1文字^{もじ}」という魔法^{まほう}が発動^{はつどう}します。
- /\. / と書^かくと、文章^{ぶんしょう}の中^{なか}にある「点^{てん} (ドット)」そのものを探^{さが}せます。

まとめテーブル (留^{りゅう}がくせいむ)

記号 ^{きごう}	意味 ^{いみ}	覚え方 ^{おぼかた} のヒント
()	グルー ^か ープ化	「みんなで1つのチー ^{ちー} ムだよ！」
[]	どれ ^{なか} か1つ	「このリス ^{だれ} トの中 ^り の誰 ^{だれ} か1人！」
{n}	ぴた ^{かい} たり n回	「回数 ^{かいすう} を予約 ^{よやく} (する)」
{n, m}	n回 ^{かい} ～m回 ^{かい}	「最 ^{さい} 小 ^{しょう} と最 ^{さい} 大 ^{だい} を決 ^き める」
\	エスケープ	「魔法 ^{まほう} をキャンセル！」

いかがでしょうか？これで正規表現の主要なパーツが揃いました。

留学生へのアドバイス

RegExpは、最初は難しくて「暗号」に見えます。でも、全部覚える必要はありません！

「RegExpは『文字の形をチェックするルールブック』だよ。
郵便番号、メールアドレス、パスワードの強さ（英数字が入っているか）を調べるときに、世界中のプログラマーが使っているんだ。
難しいパターンは、インターネットで『RegExp 郵便番号』のように検索してコピーすれば大丈夫だよ！」

[!IMPORTANT]

スラッシュ / / の後ろにつけるオプション（フラグ）

- `/abc/i`：大文字と小文字を区別しない（ignore）。
- `/abc/g`：全部探す（global）。

この2つを知ると、さらに便利に使ってもらえます。

いかがでしょうか？「完璧なパターンを自分で書く」ことよりも、「ルールで検索できる便利な道具がある」ことを知ることが第一歩です！

V. 実際に使ってみよう

1. Stringオブジェクトのmatch()メソッドと連携

上の例では、RegExpオブジェクトのtest()メソッドを使って、そのパターンが含まれているかどうかをチェックしました。

しかし、JavaScriptで文字列の中から特定の文字を探し出すときに、一番よく使われるのは、「RegExpオブジェクト（パターン）」と「Stringメソッドのmatch()」の連携という、強力な組み合わせです。

RegExpのtest()やexec()はパターンが主語でしたが、match()は文字列が主語になります。

- 書き方の形：`文字列もじれつ.match(パターン)`

例え：「この文章のなかで（文字列）、このルールに合うものを（パターン）、集めてちょうだい！（match）」

例

```
const myRegExp = new RegExp("app.*"); // 「app.*」というパターンがあるか？
const text = "I have an apple.";
```



```
mat = text.match(myRegExp);
if (mat !== null)
  console.log("見つかりました!");
  console.log(mat); // ['apple.', index: 10, input: 'I have an apple.', groups:
  undefined]
```

match()メソッドの戻り値は配列で、上の場合

- マッチした最初の文字列 (apple)
- マッチした文字の位置 (10)
- 調べた対象の文字列 ('I have an apple.')

です (groupsについての解説は省略)。

2. 正規表現の g フラグによる動きの変化

match() の本当の面白さは、正規表現の後ろにつける g (グローバルフラグ: 全部探すという意味) があるかないかで、返ってくる結果の形 (データ型) がガラリと変わることです。

g フラグをとくとき: 合うものを全部集めてリストに

文字列の中を上から下まで全部調べて、パターンに合う言葉を「ただのきれいな配列 (リスト)」にして全部返してくれます。詳しい情報は入りません。

```
const text = "私の学校には、Aクラス、Bクラス、Cクラスがあります。";

const myRegExpG = new RegExp("クラス", "g"); // うしろに g をつけました！

const resultG = text.match(myRegExpG);
console.log(resultG); // ["クラス", "クラス", "クラス"] (3つぜんぶつかまえた！)
console.log(resultG.length); // 3 (いくつ見つかったかもすぐわかります)
```

[!note] もし文字列の中にパターンに合う言葉が1つもなかった場合、match() はどちらの場合でも null (空っぽ) を返します。

3. i フラグの i は何の略?

この i は、英語の IgnoreCase (イグノア・ケース: 大文字小文字を無視する) という言葉の頭文字です。

通常、JavaScriptの正規表現は大文字と小文字を「完全に別の文字」として区別します。しかし、i フラグをつけると、それらを 同じ文字 として扱ってくれるようになります。

コードで見る違い (フラグがある時・ない時)

自分のパソコンのブランド (Apple) について書いた文章から文字を探してみましょう。

```
const text = "My computer is an apple MacBook."; // 小文字の apple

// 1. i フラグを【つけない】とき（きびしく さがす）
const regExp1 = new RegExp("Apple"); // 大文字の A
console.log(text.match(regExp1) !== null); // false （大文字と小文字がちがうので、見
つからない！）

// 2. i フラグを【つける】とき（やさしく さがす）
const regExp2 = new RegExp("Apple", "i"); // うしろに i をつけました！
console.log(text.match(regExp2) !== null); // true （大文字小文字をむしるので、見
つかった！）
```

このように、パターンのスラッシュの後ろに **/パターン/i** と書くだけで、Apple も apple も APPLE も、ぜんぶ一緒に見つけてくれるようになります。

前に習った **g** フラグと一緒に使える？

はい、一緒に使えます！ 順番は **ig** でも **gi** でもどちらでも大丈夫です。

これらを組み合わせると、「大文字小文字を気にせず（**i**）、文章全体からすべて集める（**g**）」という強力な検索ができます。

```
const text = "JavaScript is great. I love javascript!";

// 大文字小文字を気にせず（i）、ぜんぶ集める（g）
const myRegExp = new RegExp("javascript", "ig");

const result = text.match(myRegExp);
console.log(result); // ["JavaScript", "javascript"] （両方ともしっかり捕まえた！）
```

2. Replace()メソッド

JavaScriptにおいて、**RegExp**（正規表現）と **String** の **replace()** メソッドの組み合わせは、文章の中の特定の言葉を「見つけて、別の言葉に置き換える（チェンジする）」ときに最もよく使われる超重要テクニックです。

「文章（String）の中から、指定したルール（RegExp）に合う文字を見つけて、別の新しい文字に『翻訳（または交換）』する作業だよ」と教えてあげるとスムーズに伝わります。

1. 基本的な書き方

replace() メソッドは、文字列（String）が主語になります。第1引数に「正規表現（パターン）」を、第2引数に「新しく置き換えたい文字」を書きます。

- 書き方の形： **文字列.replace(正規表現パターン, "新しい文字")**

- **注意点**：もとの文字列（変数）の中身を直接書き換えるのではなく、「置き換えた後の新しい文字列」を新しく作って返してくれる（戻り値にする）性質があります。

2. フラグ（**g** や **i**）を組み合わせたときの動きの違い

前に学んだフラグを使うことで、文字の置き換え方がガラリと変わります。ここが留学生のコードの楽しさであり、つまずきやすいポイントでもあります。

今回は、文章の中の「バナナ（banana）」を「りんご（apple）」に置き換える例で見てみましょう。

① フラグを【何もつけない】とき：最初の1つだけ置き換える

```
const text = "私は Banana が好きです。でも、banana はもっと好きです。";

// フラグなし：大文字（おおもじ）小文字（こもじ）を区別（くべつ）し、最初（さいしょ）の1つだけ
const pattern1 = new RegExp("banana");

const result1 = text.replace(pattern1, "りんご");
console.log(result1);
// 結果（けっか）："私は Banana が好きです。でも、りんご はもっと好きです。"
// ※ 小文字の "banana" だけが置（お）き換（か）わり、最初にある大文字の "Banana" は無視（むし）されました。
```

② **i** フラグ（大文字小文字を無視）をつけたとき

```
const pattern2 = new RegExp("banana", "i"); // 大文字小文字を気にしない

const result2 = text.replace(pattern2, "りんご");
console.log(result2);
// 結果："私は りんご が好きです。でも、banana はもっと好きです。"
// ※ 最初に見つかった大文字の "Banana" が置（お）き換（か）わりましたが、2つ目の小文字はそのままです。
```

③ **g** と **i** フラグを【両方】つけたとき：全部まとめて置き換える！

文章の中にあるターゲットを、大文字小文字に関わらずすべて一発で置き換えたいときは、**gi**（または **ig**）を使います。これが実務でも一番よく使われる形です。

```
const pattern3 = new RegExp("banana", "gi"); // 大文字小文字を気にせず、ぜんぶ！

const result3 = text.replace(pattern3, "りんご");
console.log(result3);
```

```
// 結果: "私は りんご が好きです。でも、りんご はもっと好きです。"
// ※ みごとにすべてのバナナがりんごに変身（へんしん）しました！
```

3. 実務でよくある使い方：「NGワード（不適切な言葉）」の非表示

Webサイトの掲示板やチャットアプリを作るとき、誰かが悪口や秘密の情報を書き込んだら、自動的に *** や [<ruby>削除<rt>さくじょ</rt></ruby>] に置き換えるシステム（フィルター機能）によく使われます。

```
const chatMessage = "お前のバカ！BAKA！本当にもう知らない！";

// 「バカ」という言葉（ことば）を大文字小文字にかかわらず（gi）すべて見つける
const ngWordPattern = new RegExp("バカ|baka", "gi")

// 安全な文字に置き換える
const safeMessage = chatMessage.replace(ngWordPattern, "***");
console.log(safeMessage);
// 結果: "お前の***！***！本当にもう知らない！"
```

4. まとめ

String の `replace()` メソッドは、正規表現と一緒に使うことで、文章の中の文字を自由自在《じゅうじざい》に書き換えることができます。
大切なルールは次の3つです。

1. 順番：文字列.`replace(/探すパターン/, "新しい文字")` の順番で書きます。
2. フラグのパワー：/パターン/gi のように g と i を一緒につけると、大文字・小文字を気にせず、文章の中にあるターゲットを**全部**まとめて**新しい文字**に変えてくれます。
3. 元の文字は消えない：`replace()` を使っても、元の変数の中身は変わりません。必ず新しく作った結果を別の変数（`const newText = ...`）に代入《だいにゅう》して使いましょう。

ユーザーが入力したデータから、いらないスペース（空白）を消したり、危険な文字を隠したりするときにとっても役に立つので、ぜひ覚えてくださいね！